LAB 03
Parallel Programming on PULP

Alberto Dequino (alberto.dequino@unibo.it)

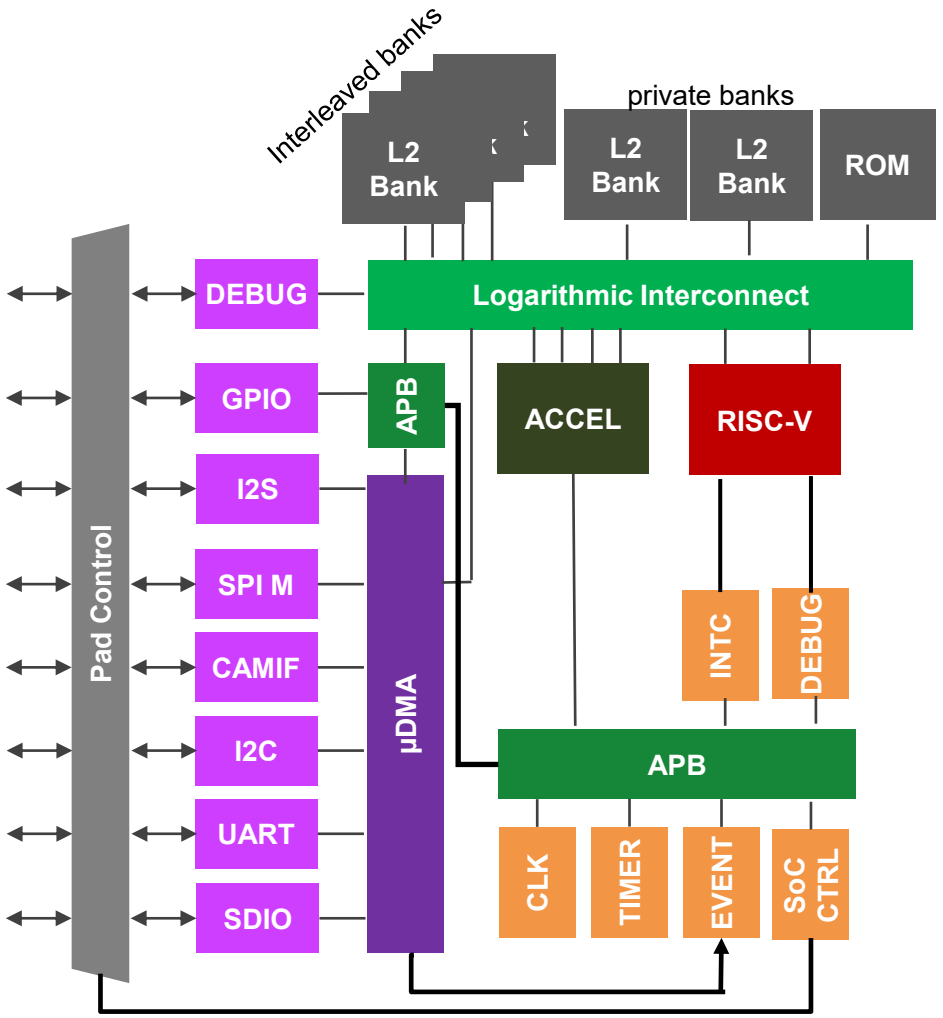
Călin Diaconu (calin.diaconu2@unibo.it)

Francesco Conti

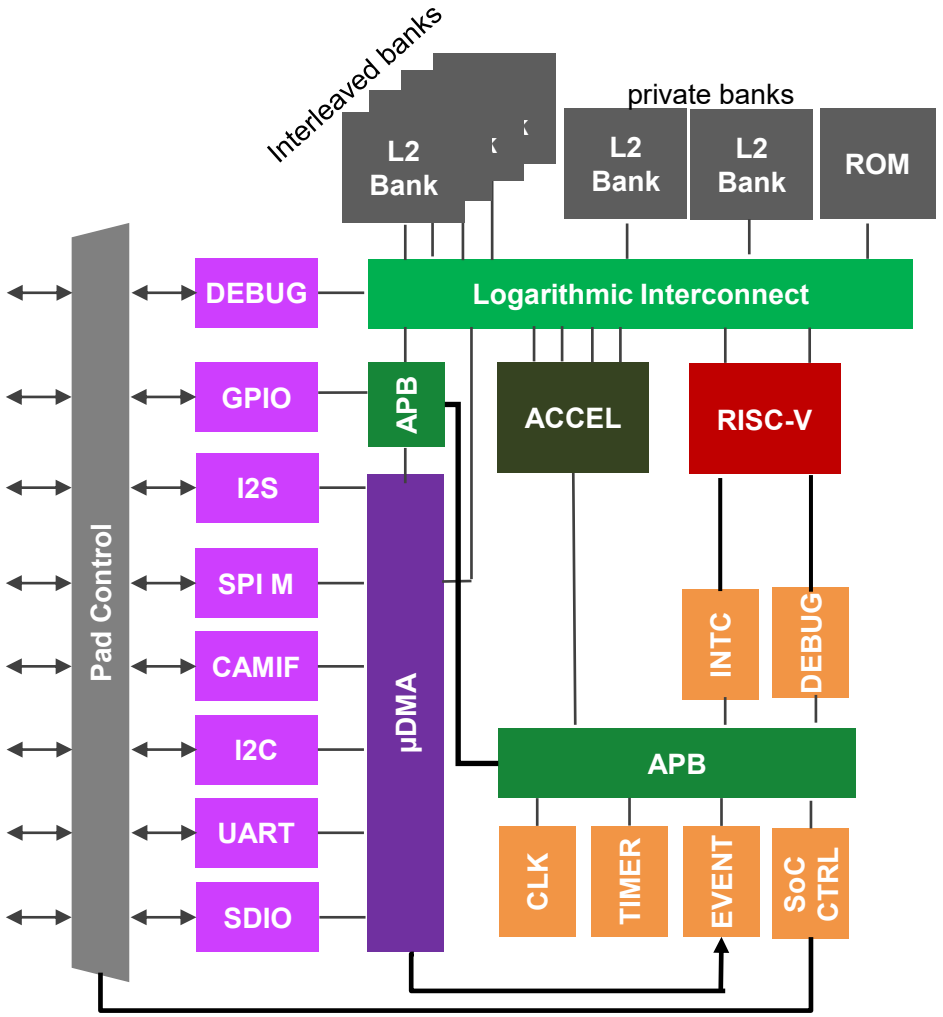
Giuseppe Tagliavini

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

PULP: A Parallel Ultra-Low-Power Computing Platform

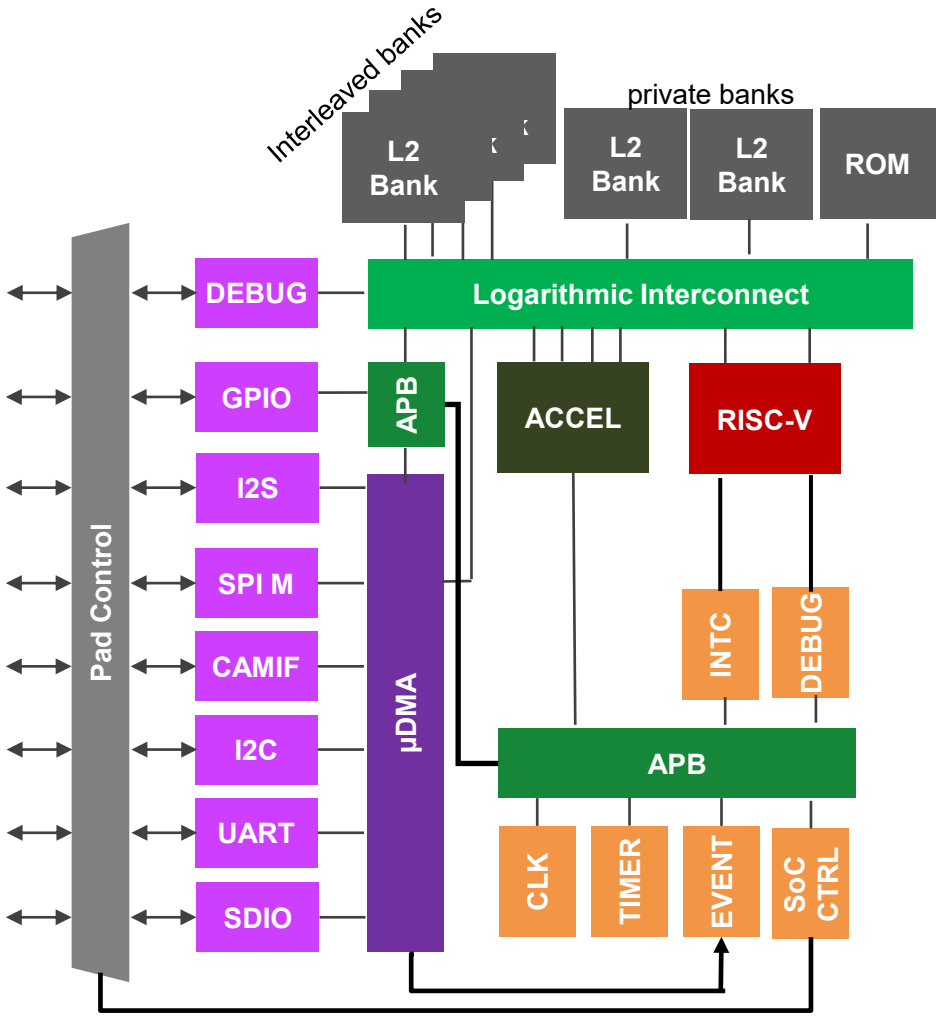


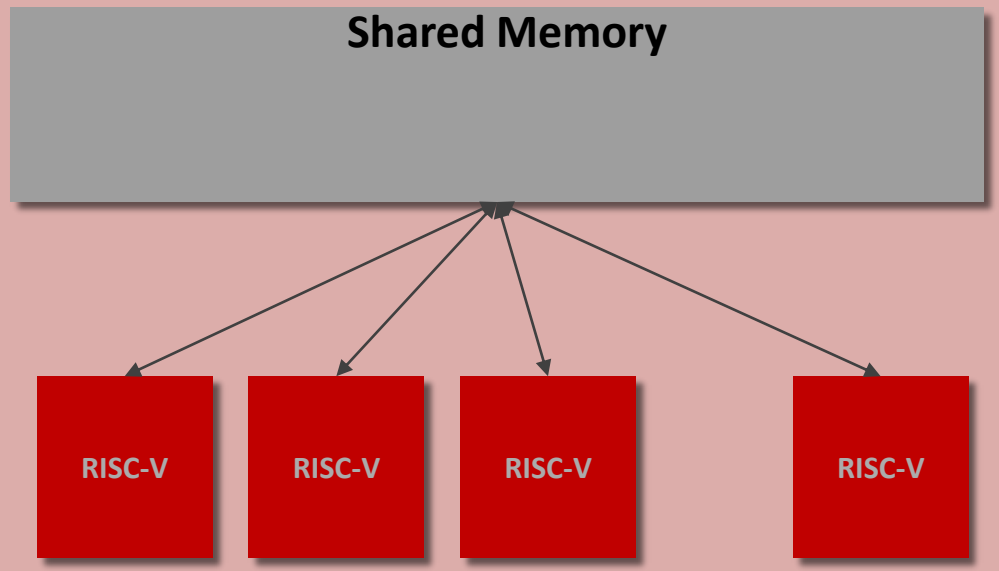
PULP: A Parallel Ultra-Low-Power Computing Platform



?

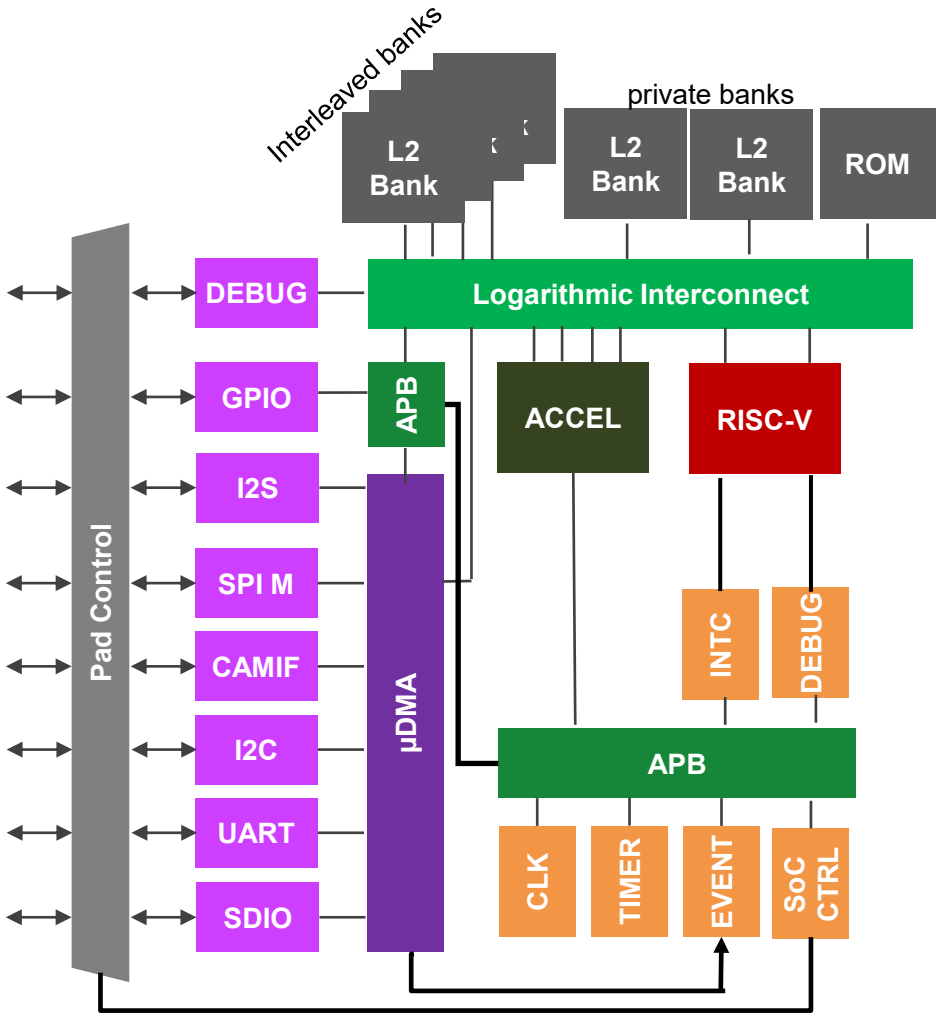
PULP: A Parallel Ultra-Low-Power Computing Platform

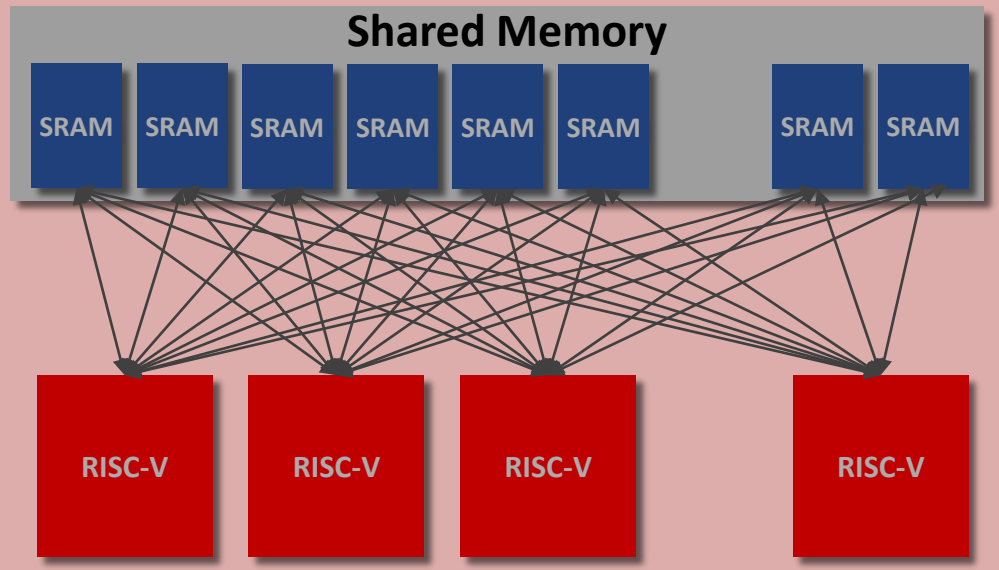




Target a **Shared-Memory** parallel programming model: 4—16 DSP cores accessing a **Shared L1 Memory** (***Tightly Coupled Data Mem*** or ***TCDM***)
How is memory **organized** to allow this?

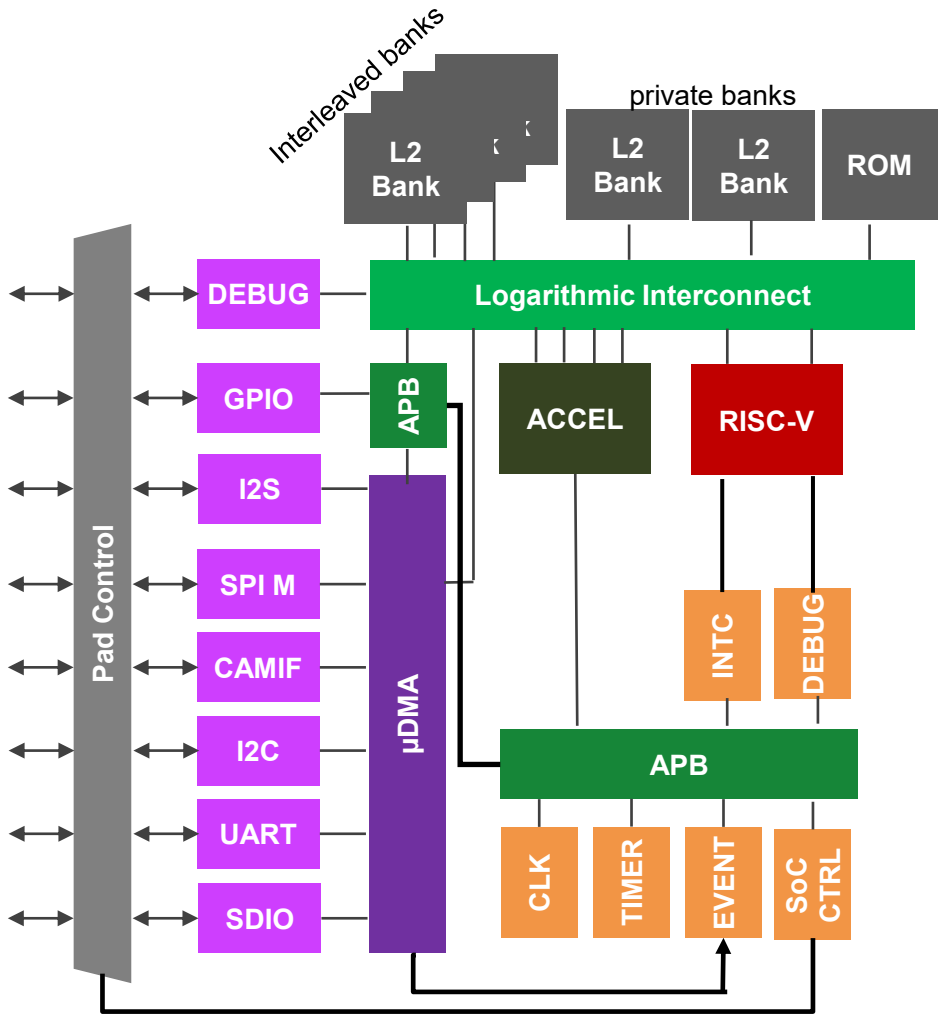
PULP: A Parallel Ultra-Low-Power Computing Platform

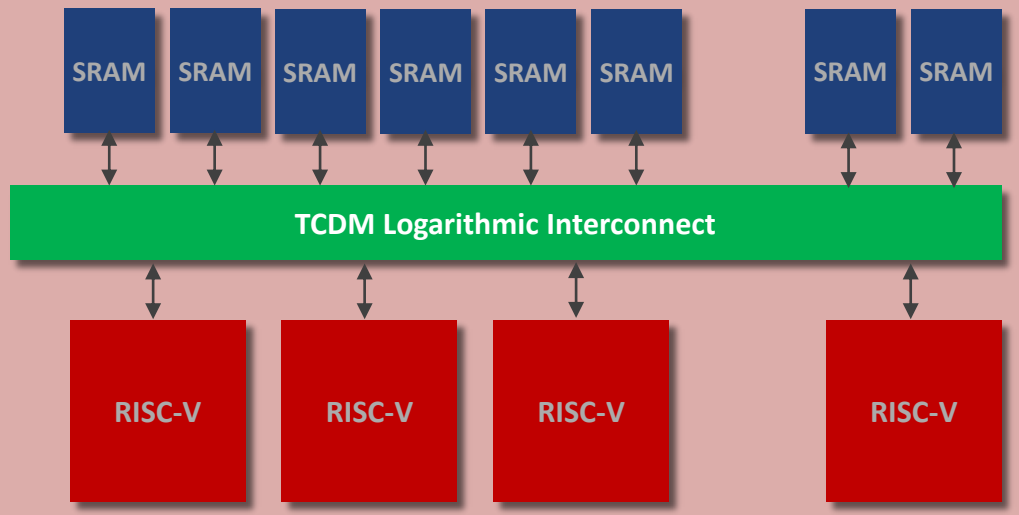




Target a **Shared-Memory** parallel programming model: 4—16 DSP cores accessing a **Shared L1 Memory** (***Tightly Coupled Data Mem*** or ***TCDM***)
How is memory **organized** to allow this?
The memory is organized in **Multiple Banks** → **concurrent access**.

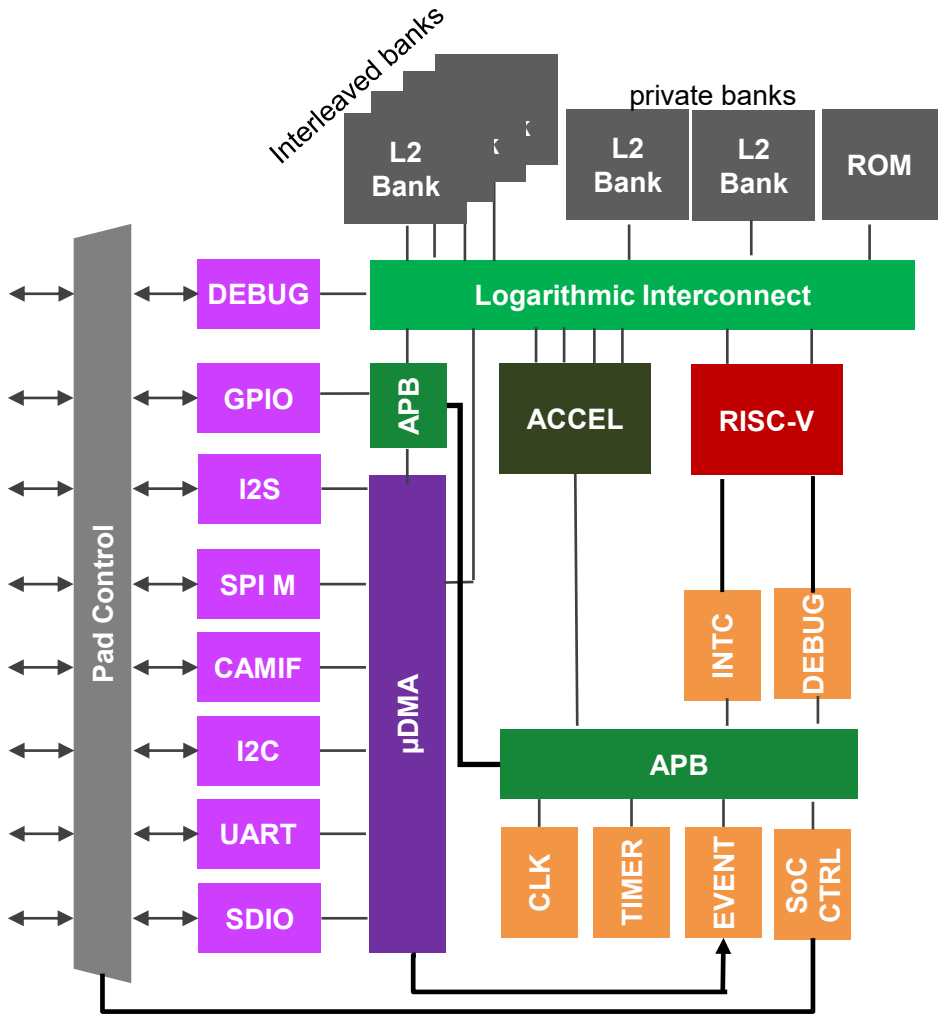
PULP: A Parallel Ultra-Low-Power Computing Platform

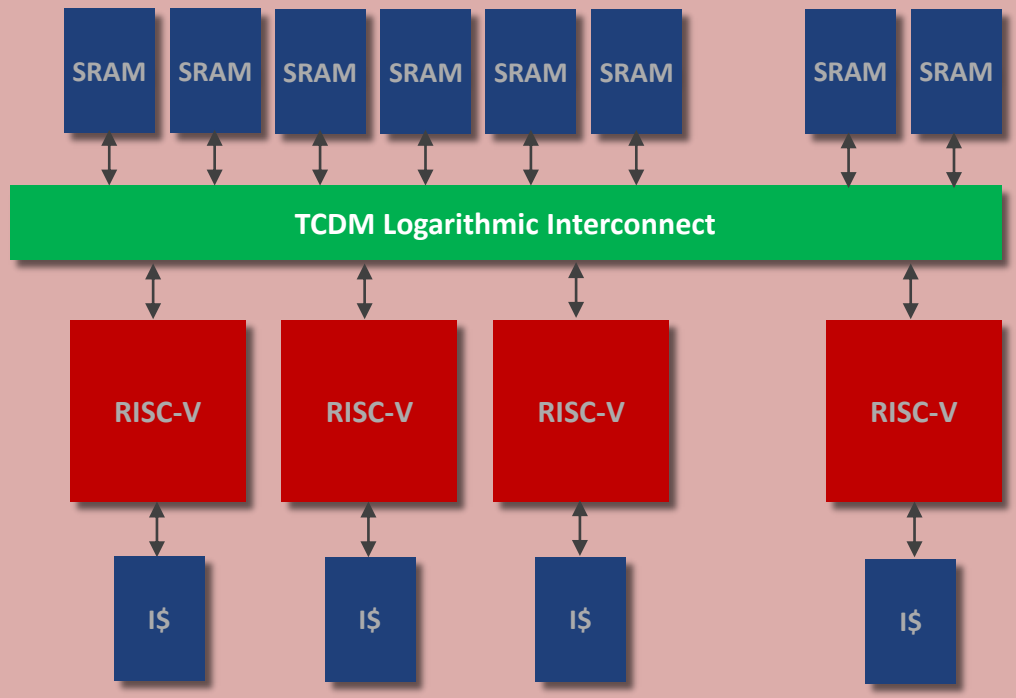




Target a **Shared-Memory** parallel programming model: 4—16 DSP cores accessing a **Shared L1 Memory** (***Tightly Coupled Data Mem*** or ***TCDM***)
How is memory **organized** to allow this?
The memory is organized in **Multiple Banks** → **concurrent access**.
Solution: **full connectivity** with 1-cycle latency **crossbar**

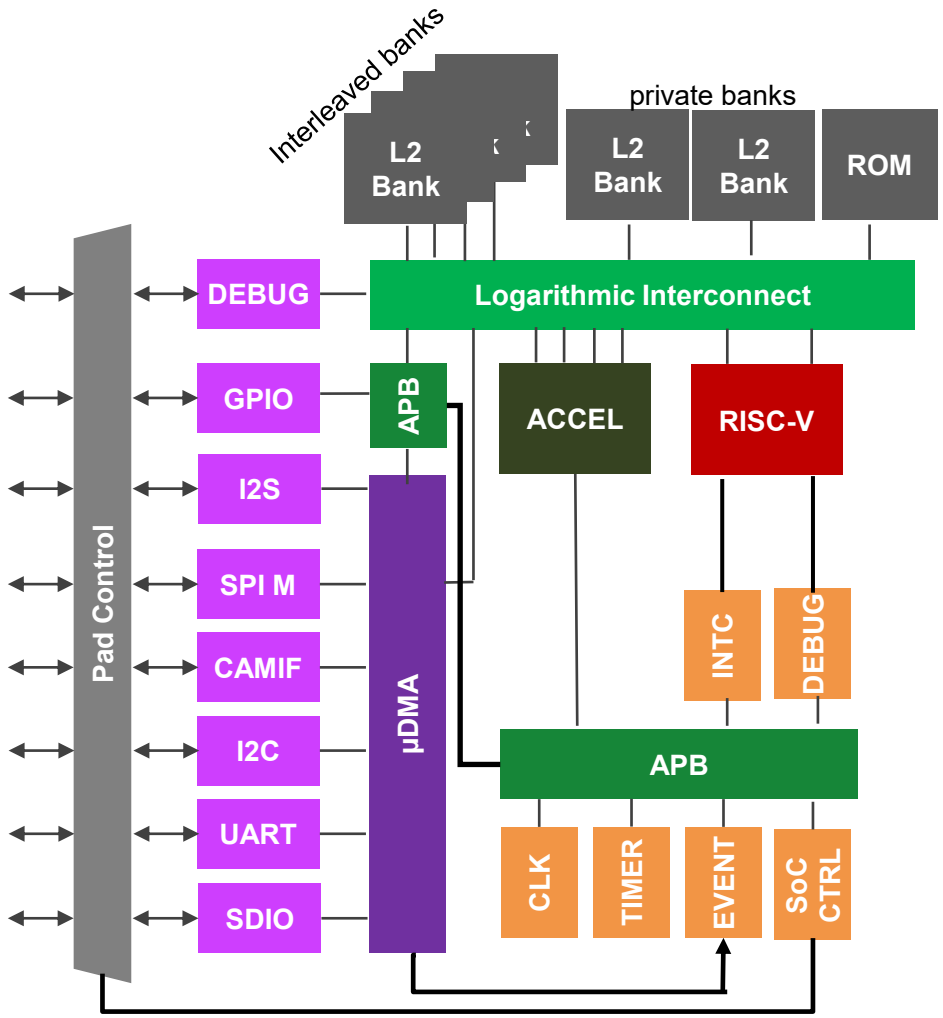
PULP: A Parallel Ultra-Low-Power Computing Platform

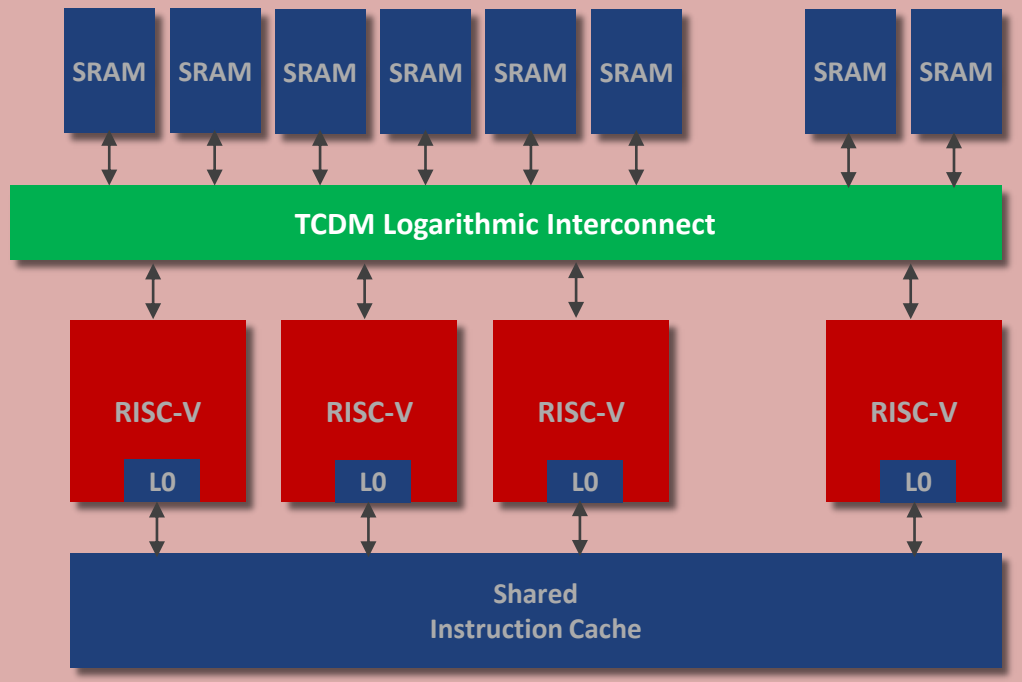




Multiple separate instruction streams (hardware threads):
MIMD (Multiple Instruction Multiple Data) execution scheme...

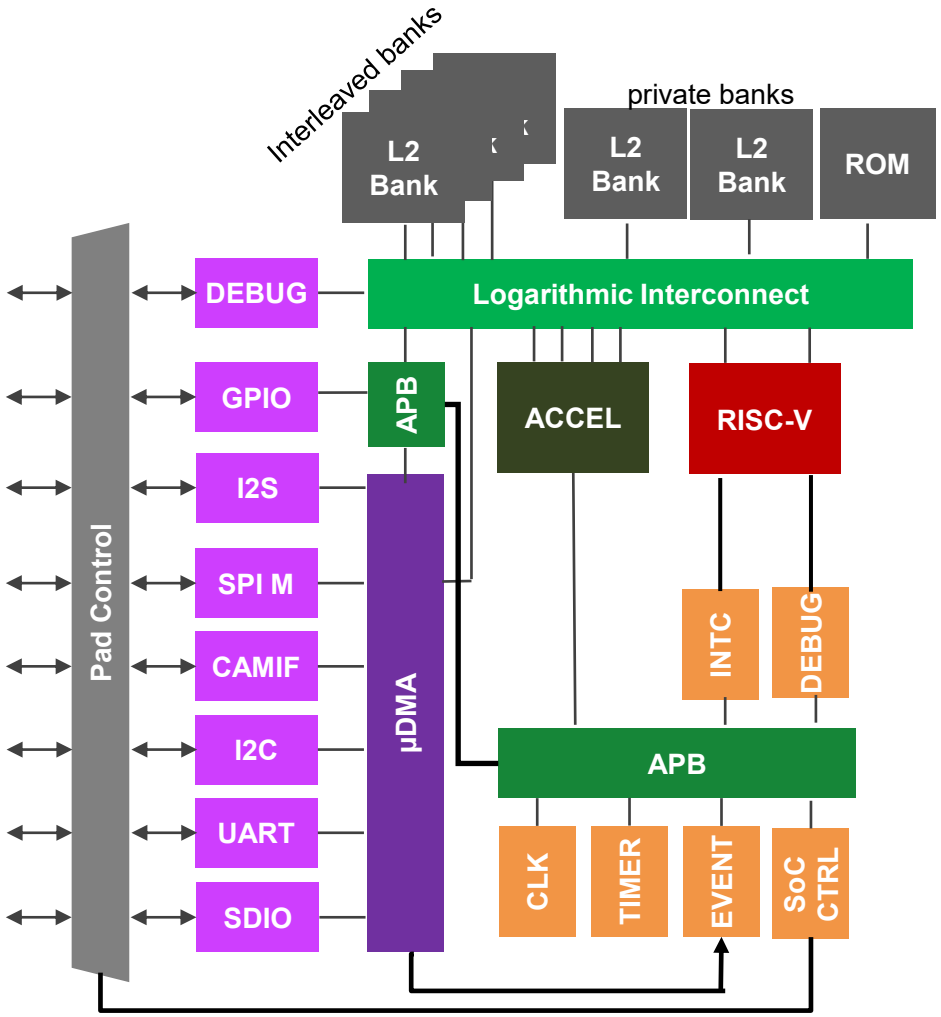
PULP: A Parallel Ultra-Low-Power Computing Platform

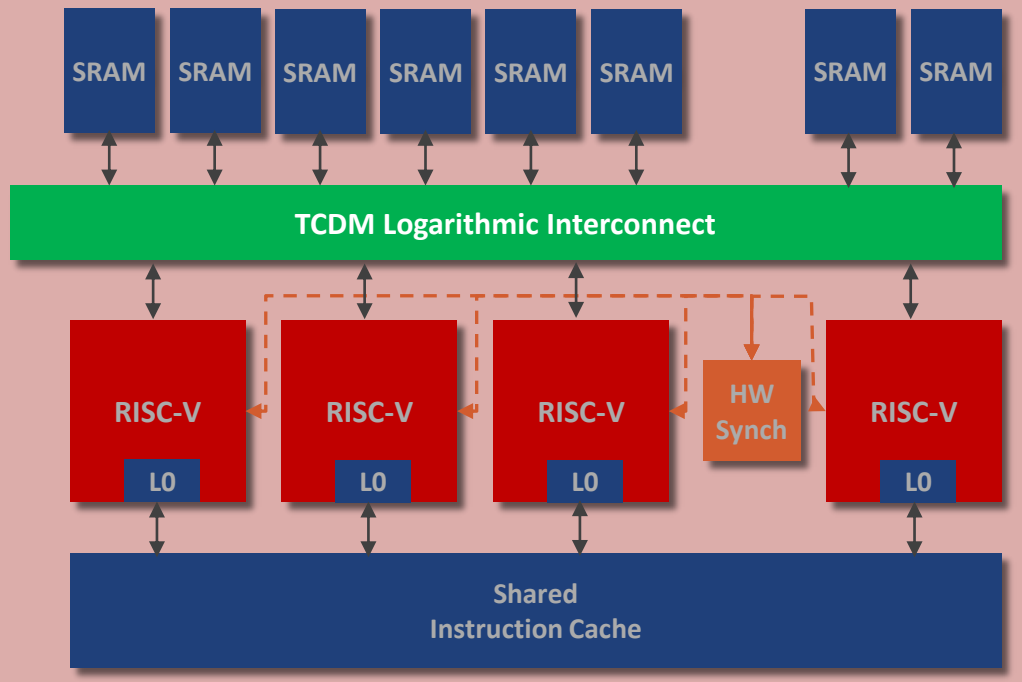




Multiple separate instruction streams (hardware threads): **MIMD** (Multiple Instruction Multiple Data) execution scheme... but optimized for a single parallel program (**SPMD**)

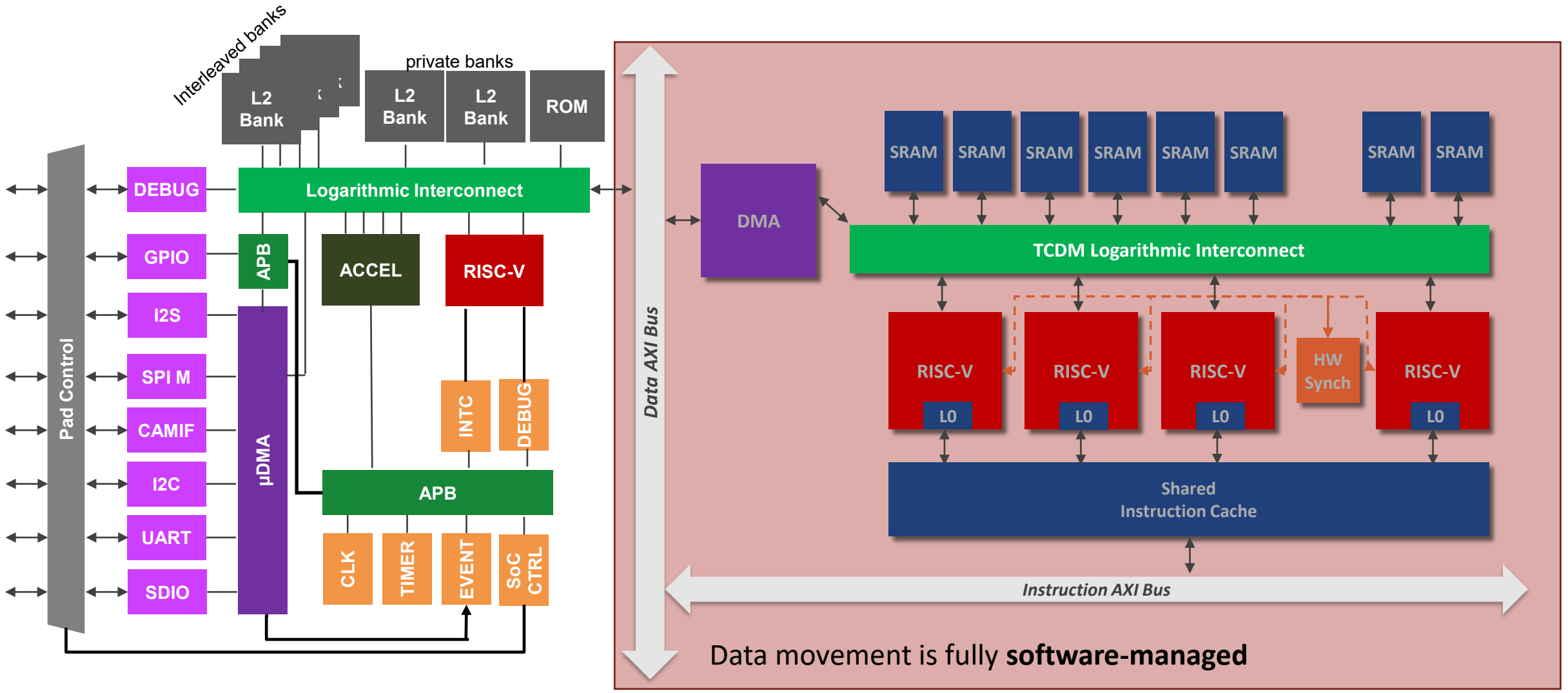
PULP: A Parallel Ultra-Low-Power Computing Platform





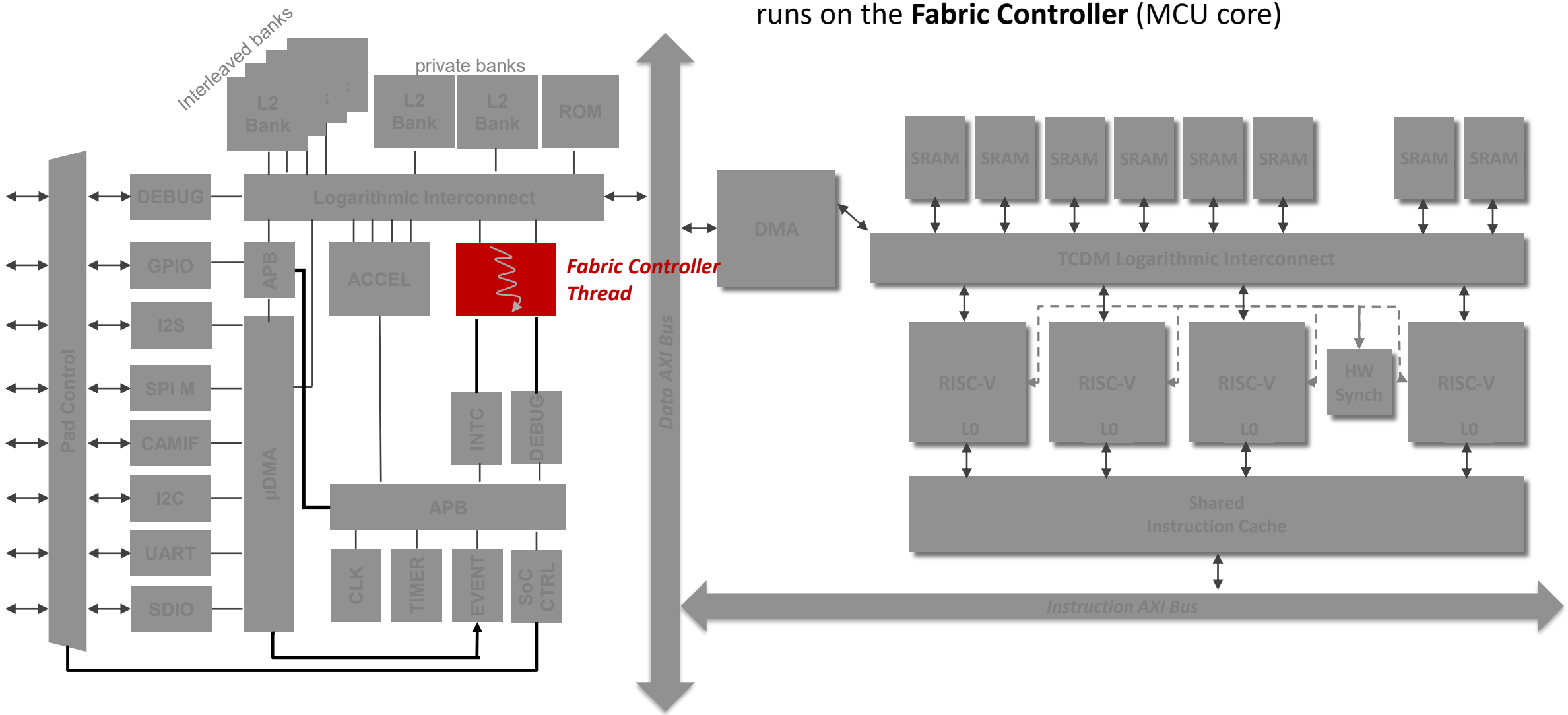
High-speed synchronization to minimize overhead in **synchronization primitives**

PULP: A Parallel Ultra-Low-Power Computing Platform



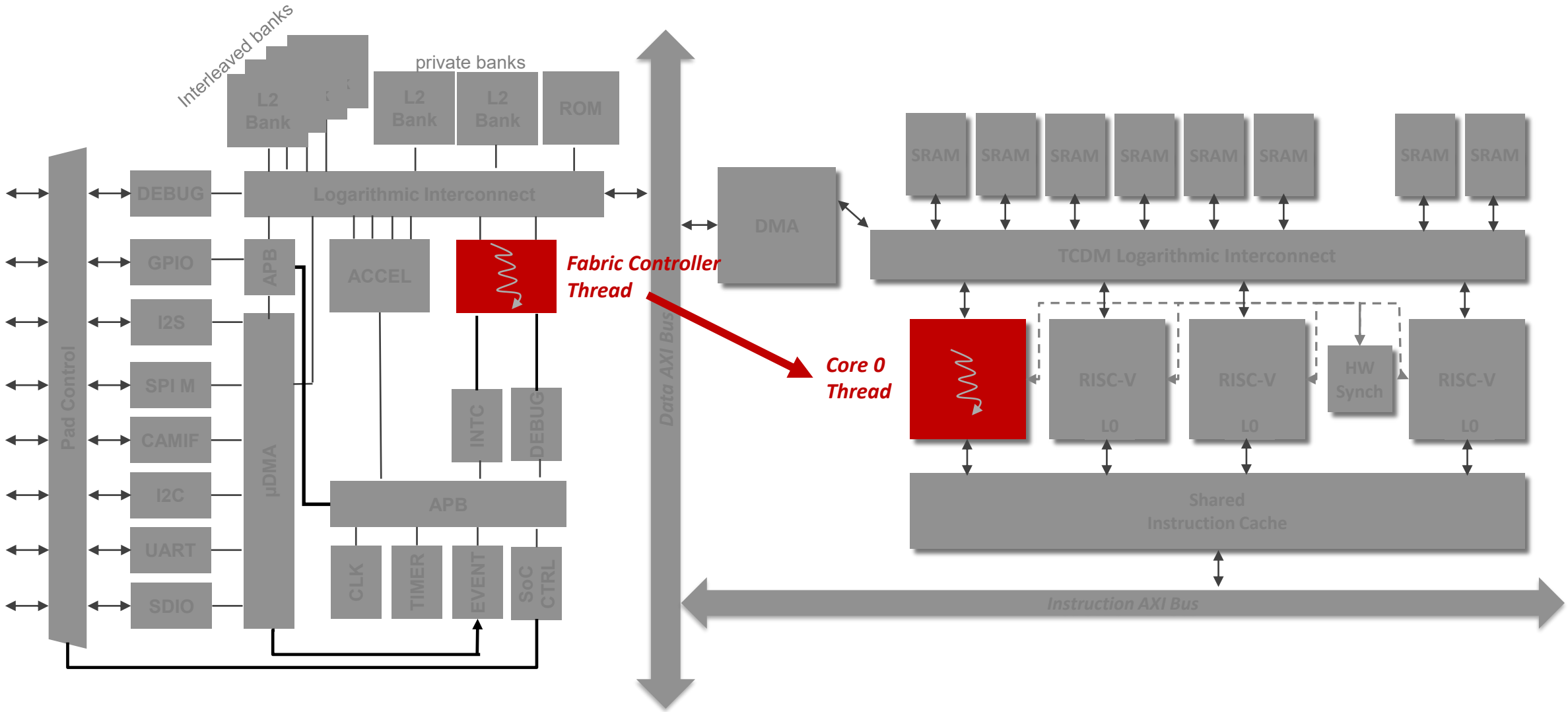
PULP Execution Model

The Cluster is **clock-gated** at boot: a single thread runs on the **Fabric Controller** (MCU core)



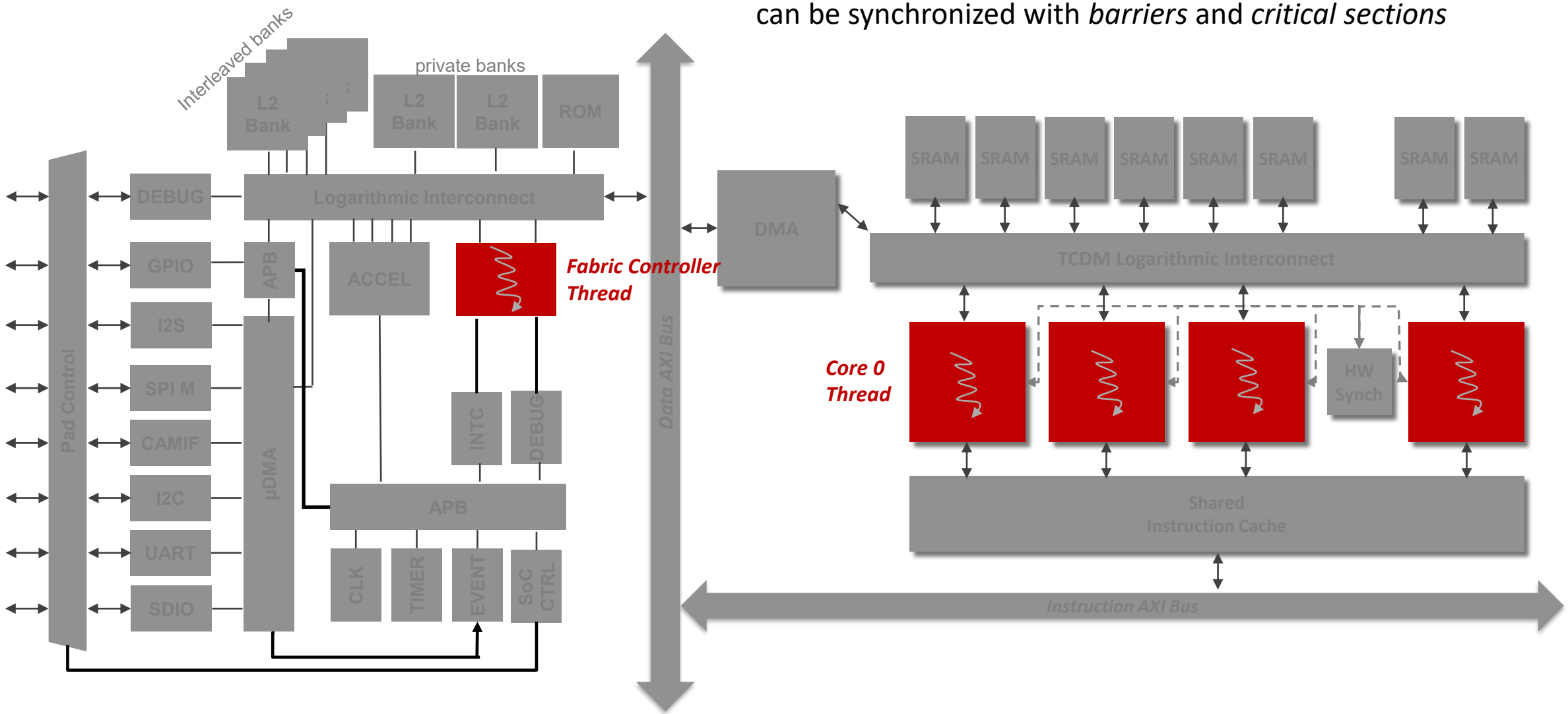
PULP Execution Model

Activate the cluster and **call** a function on the **cluster core 0**



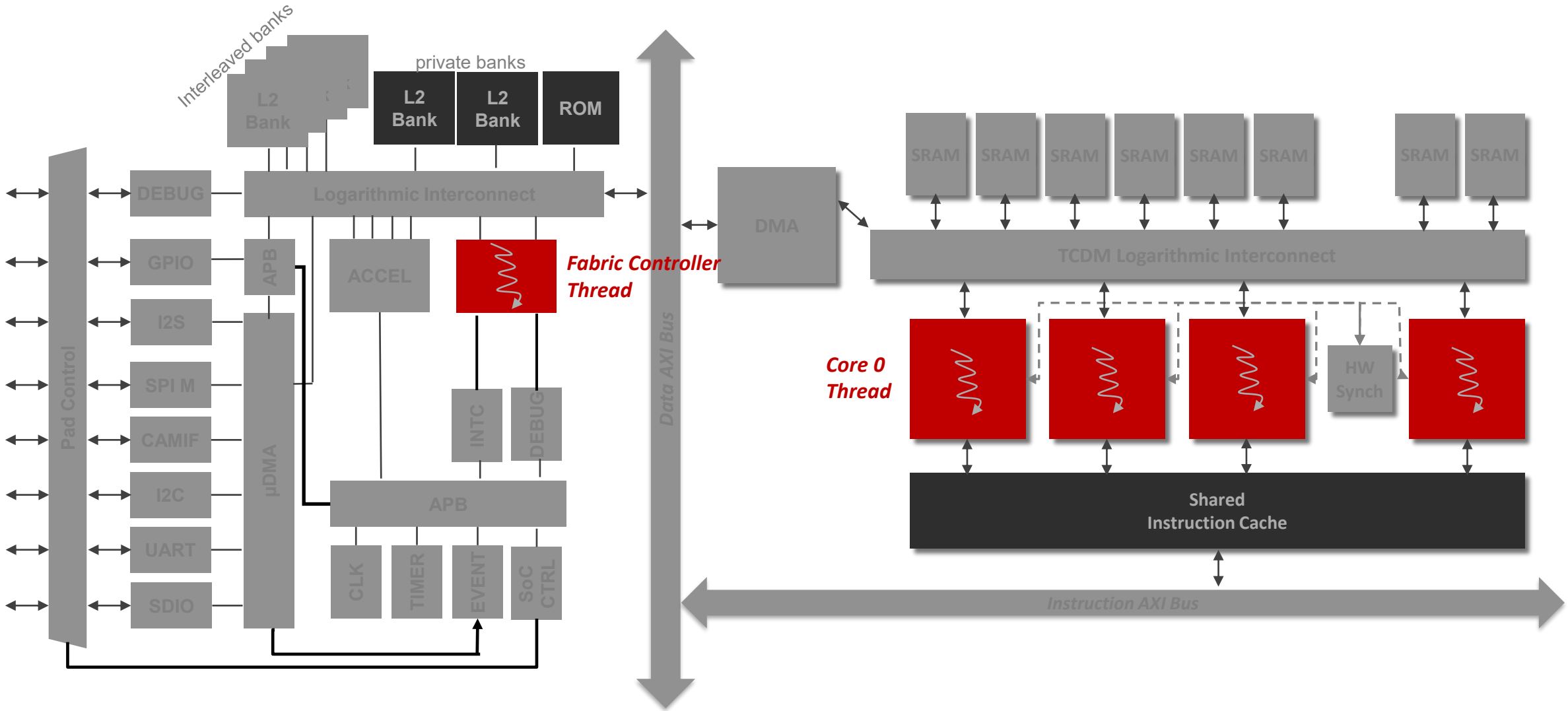
PULP Execution Model

Fork an **execution team** on multiple cluster cores; they can be synchronized with *barriers* and *critical sections*



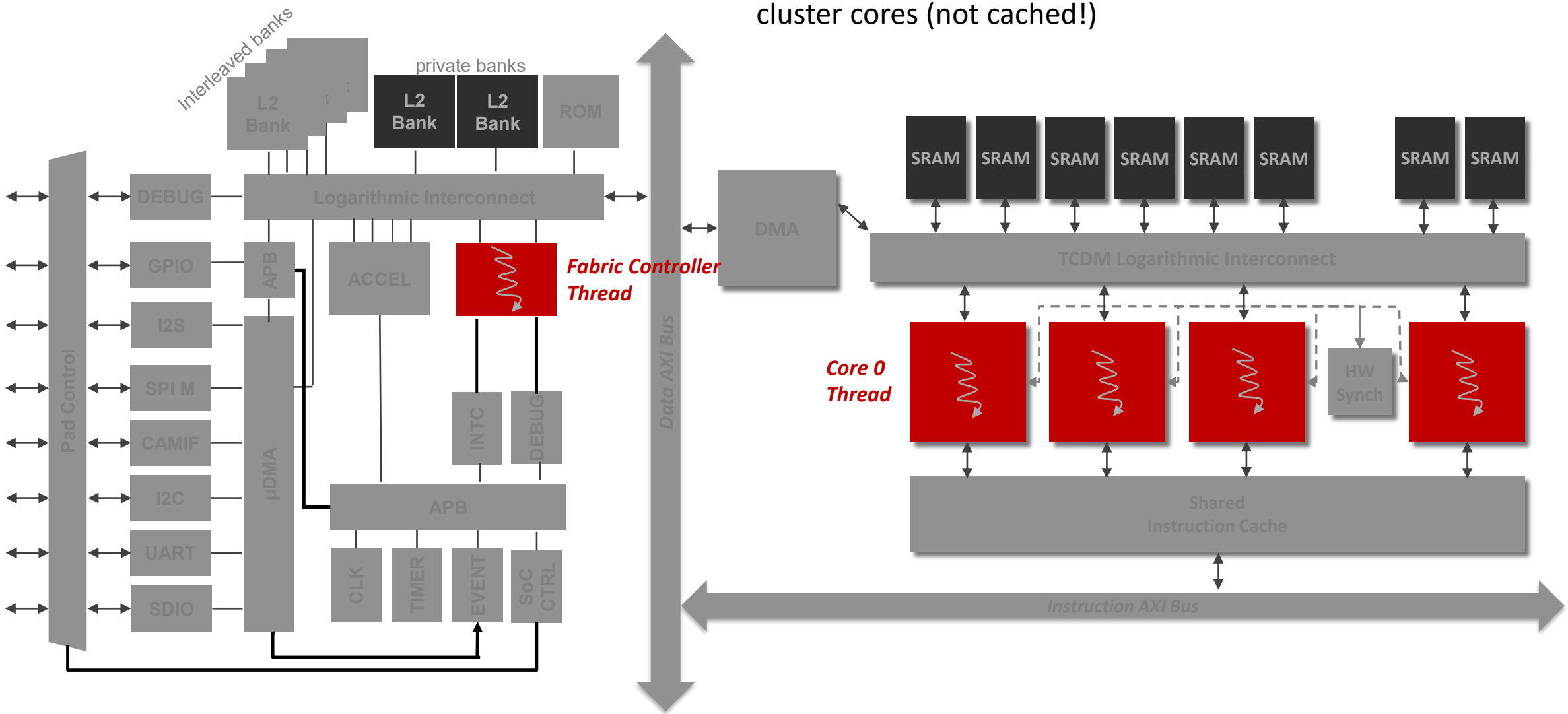
PULP Execution Model

All code is in **L2 memory** (cached for cluster threads)

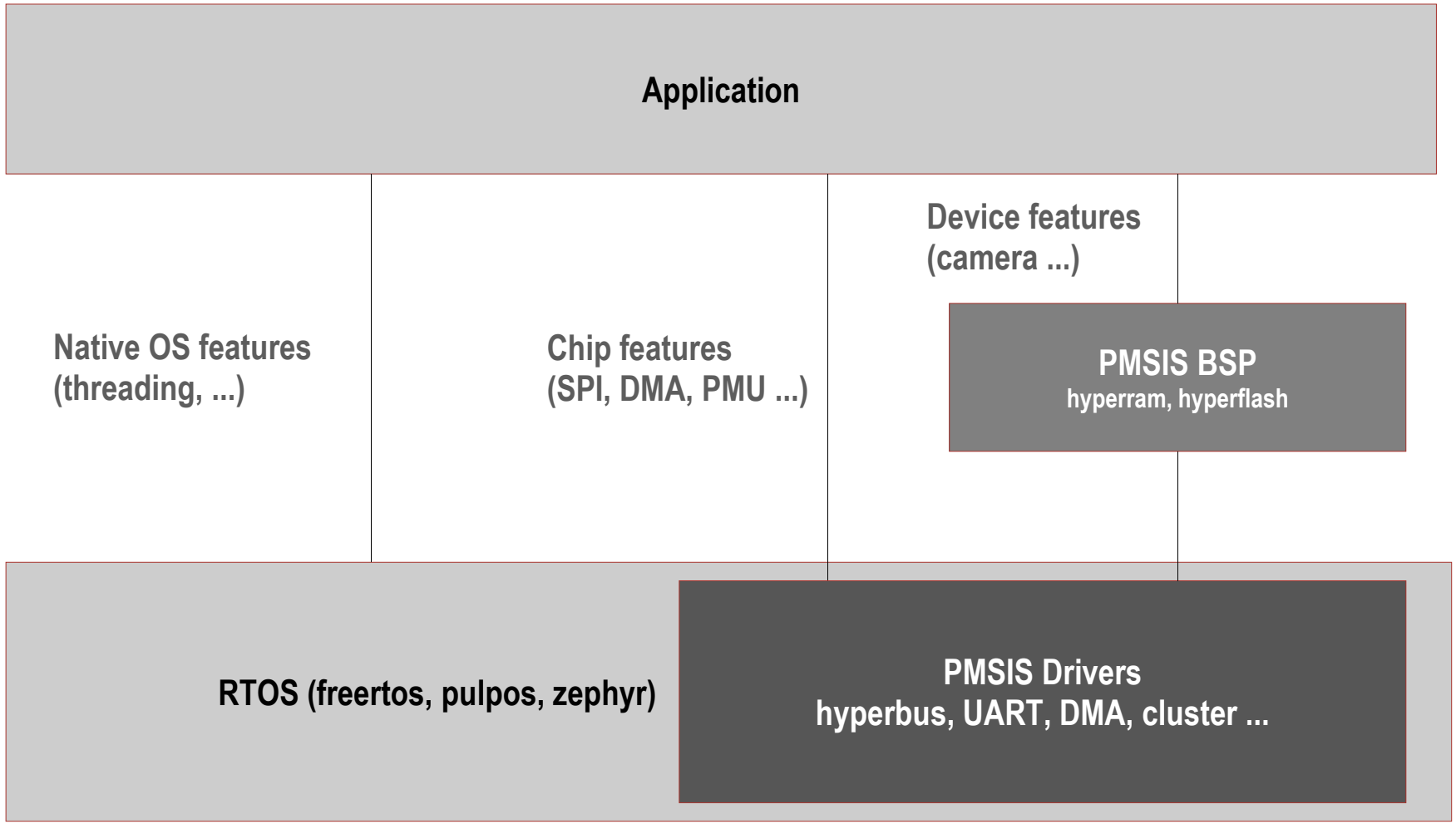


PULP Execution Model

Stacks are in L2 for the FC, and in **L1** (**TCDM**) for the cluster cores (not cached!)



PULP Microcontroller Software Interface Standard (PMSIS)



PMSIS: How to Manage a Device Lifecycle

- **Configuration and initialization (*device specific*)**

```
conf_init()
open_from_conf()
```

- **Prepare the device for usage:**

```
open()
```

- **Perform required operations (*device specific*)**

- **Release the resources**

```
close()
```

PMSIS: Using the Cluster as a Device

```
struct pi_device cluster_dev = {0};
struct pi_cluster_conf conf;
struct pi_cluster_task cluster_task = {0};

// task allocation
pi_cluster_task(&cluster_task, cluster_entry, NULL);

// init the cluster
pi_cluster_conf_init(&conf);
pi_open_from_conf(&cluster_dev, &conf);

// open the cluster
if (pi_cluster_open(&cluster_dev)) return -1;

// offload an entry point to the cluster
pi_cluster_send_task_to_cl(&cluster_dev, &cluster_task);

// releasing the cluster
pi_cluster_close(&cluster_dev);
```

Function pointer

PMSIS data structures

Create a task for the cluster

Initialize the cluster device

Open the cluster device

Execute code on the cluster

Release the cluster device

Executing on the Cluster: Fork/join + SPMD

```
static void cluster_entry(void *arg)
{
  printf("Hello from cluster\n");
  // ...

  pi_cl_team_fork(NUM_CORES, cluster_fn, (void *) &args);

  // ...
}
```

Data pointer

Function pointer

Executed on core 0

Fork the execution of the same function on NUM_CORES cores

Executed on core 0

Synchronization: Barriers and Critical Sections

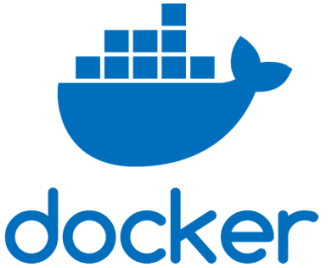
- **Barriers (*used for intermediate and final join points*):**

```
pi_cl_team_barrier();
```

- **Critical sections (*used to avoid critical races*):**

```
pi_cl_team_critical_enter();
// code in the critical section
pi_cl_team_critical_exit();
```

Hands-on Part, LAB 01 Recap: Getting Started: *Docker setup*



- **How to install:**
  - Install Docker Desktop* - https://www.docker.com/products/docker-desktop/

    *For Windows users, it's recommended to use it through Windows Subsystem for Linux - WSL (install instructions); otherwise, it would become quite computationally-intensive, since it runs through a virtual machine.

  - Install VS Code https://code.visualstudio.com/

  - In VS Code, install the Dev Containers extension: https://marketplace.visualstudio.com/items?itemName=ms-vscode-remote.remote-containers

Hands-on Part, LAB 01 Recap:
Getting Started: *Open project in the Docker container*

First steps:

1. Clone* this lab's project locally (in WSL, if using it)

```
$ git clone https://github.com/EEESlab/HSDES-LAB04-PULP_Parallel_Bare.git
```

2. Enter the newly created local directory of the project

```
$ cd HSDES-LAB04-PULP_Parallel_Bare
```

3. Open the project in VS Code

```
$ code .
```

At this moment, the project is open in your local "environment".
We need to open it in the Docker container. For this, in VS Code:

1. **Press CTRL+Shift+P (open command palette)**
2. **Type in *"Dev Containers: Reopen in Container"* and press *Enter***

We are now inside the container. From now on, when opening VS Code, the project will automatically open inside the Docker container.
*** We will also need to use the terminal from VS Code (CTRL+Shift+`) from now on, since it is the one accessing inside the running container!**

*In these labs we will always use Git. It is a popular tool, so a useful skill to have. Here is a good intro tutorial.

Exercise 01: Vector Addition

- **Now, take a look at the code and run the function on the cluster!**

```
$ cd 01_vectAddPar
$ make clean all run CORES=1
```

➢ How many clock cycles does it take SINGLE core ?

Ex. 01 - Method #1

01_vectAddPar/kernels.c

```
void vectAddPar(int * pSrcA, int  * pSrcB, int * pDstC, int n) {
    int i, core_id;

    core_id = pi_core_id();
    for (i = core_id; i < n; i += NUM_CORES) {
        pDstC[i] = a + b;
    }


    pi_cl_team_barrier();
}
```

Distribute single iterations among the available cores

Stop the execution once all cores have reached this point

- **With NUM_CORES == 4 and n == 16:**
- **Iterations:**
- **Core id:**

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3

Ex. 01 - Method #2 - balanced workload

01_vectAddPar/kernels.c

```
void vectAddPar(int * pSrcA, int  * pSrcB, int * pDstC, int n) {
    int i, core_id, i_start, i_end, i_block, i_chunck;
    core_id = pi_core_id();
    i_chunk = n / NUM_CORES;
    i_start = core_id * i_chunk;
    i_end   = i_start + i_chunk;

    for (i = i_start; i < i_end; i++) {
       pDstC[i] =  pSrcA[i] + pSrcB[i];
    }
    pi_cl_team_barrier();
}
```

Split the wokload into adjacent blocks (*chunks*) and compute their bounds [i_start, i_end)

- **With NUM_CORES == 4 and n == 16:**
- **Iterations:**

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

- **Core id:**

0	0	0	0	1	1	1	1	2	2	2	2	3	3	3	3

Ex. 01 - Method #2 - generic workload

01_vectAddPar/kernels.c

```
void vectAddPar(int * pSrcA, int  * pSrcB, int * pDstC, int n) {
    uint32_t i, core_id, i_chunk, i_start, i_end, i_block;
    core_id = pi_core_id();
    i_chunk = (n + NUM_CORES - 1) / NUM_CORES;
    i_start = core_id * i_chunk;
    i_end   = i_start + i_chunk < n? i_start + i_chunk : n;

    for (i = i_start; i < i_end; i++) {
       pDstC[i] =  pSrcA[i] + pSrcB[i];
    }
    pi_cl_team_barrier();
}
```

1. Integer division with rounding towards infinity (1)
2. Check the upper bound of the iteration space

$$(1)\ \left\lceil \frac{a}{b} \right\rceil = \frac{a+b-1}{b}$$

- **With NUM_CORES == 4 and n == 18:**
- **Iterations:**

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

- **Core id:**

0	0	0	0	0	1	1	1	1	1	2	2	2	2	2	3	3	3

Ex. 01 - Parallel Vector Add - Try on your own

- **Apply the parallelization strategies by adapting the *vectAddPar* function.**
- **Run the code with different values of CORES = {2, 4, 8}**
- **Make sure the checksum output is correct.**
- **Measure the number of clock cycles and compute the parallel speedup=** $\frac{time\ seq}{time\ par}$

```
$ make clean all run CORES=2
$ make clean all run CORES=4
$ make clean all run CORES=8
```

Exercise 02 - Parallelize a Matrix Multiplication Kernel

- **This example is a "template" containing the code to run a matrix multiplication on the cluster. However, this is not optimized:**

```
$ cd ../02_matrixMulPar
$ make clean all run CORES=1
```

- **Your task is to parallelize the code, using methods similar to the previous exercise.**
- **Measure the speed up and collect the performance statistics!**
- **Is the speed up ideal? If not, why?**

Exercise 03: Parallelize an algorithm with data dependencies

1. **Create a new application using the previous exercises as templates, and rename it "03_computePi"**
2. **Test the sequential code and print the result!**

```
float iterative_pi(int n) {
  float x, area=0.0f;
  int i;
  for(i=0; i<n; i++) {
    x = (i + 0.5f)/n;
    area += 4.0f/(1.0f + x*x);
  }
  return area/n;
}
```

3. **Parallelize the** `iterative_pi` **kernel.**
   **Hint: Make use of the critical section to update the* `area` *variable!*

*References on Makefile rules

- **The Makefile is a "recipe" used to call the compiler/linker and, in this case, also to run the program. You can also chain several rules (e.g., `make clean all run`) and pass options (e.g., `runner_args="--vcd"`)**
- **Remove any existing build**

```
make clean
```

- **Build program (calling compiler + linker)**

```
make all
```

- **Run the program**

```
make run
```

- **Run options: you can change them by adding `runner_args="OPTIONS"`**

```
make run runner_args="--trace=.*insn.*" > trace.log   # written trace of instructions
```

- **Disassembling:**

```
# Disassemble without inlined source code:
make dis > test.S
# Disassemble with inlined source code:
/pulp/pkg/pulp_riscv_gcc/bin/riscv32-unkown-elf-objdump -D -S > test.S
```

Alberto Dequino – alberto.dequino@unibo.it

Călin Diaconu – calin.diaconu2@unibo.it

DEI – Università di Bologna
Viale del Risorgimento 2
Bologna, Italy